

Support Vector Machines

Overview

Agha Ali Raza



Sources

- StatQuest by Josh Starmer, Support Vector Machines, Clearly Explained!!!
<https://www.youtube.com/watch?v=efR1C6CvhmE&t=439s>

Children

Adults



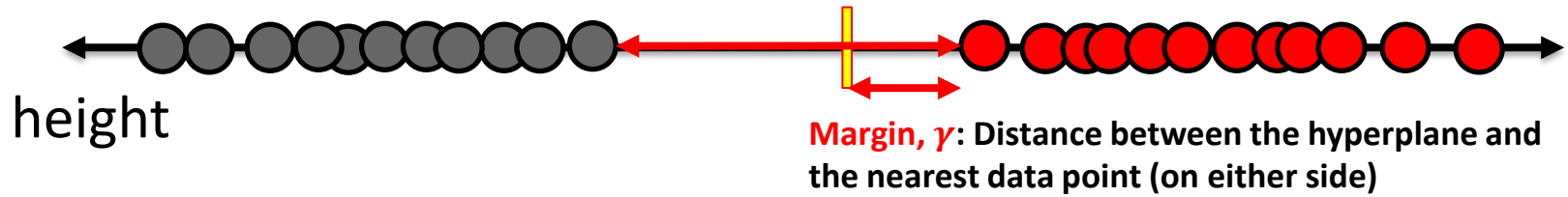
Children

Adults

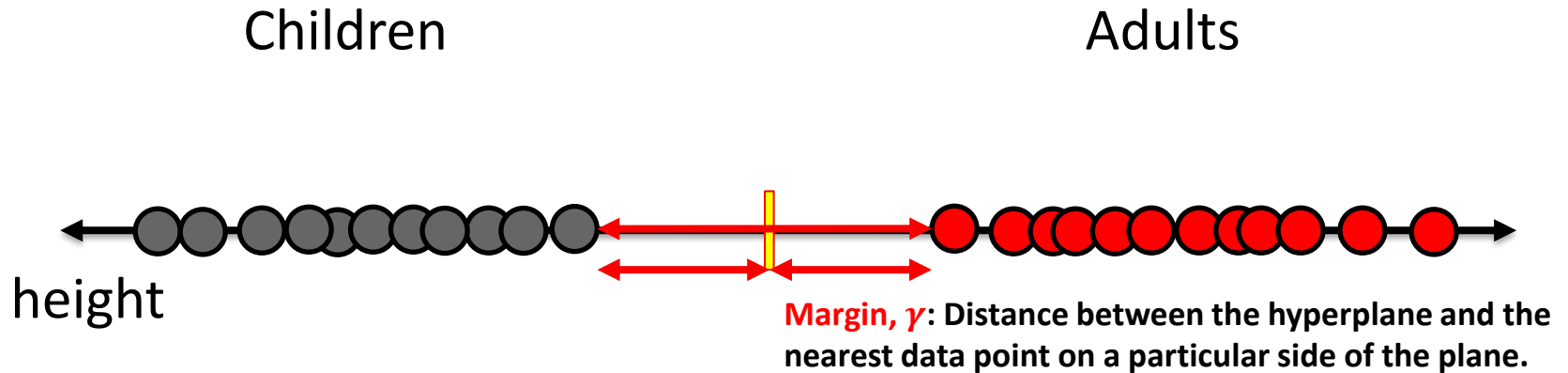


Children

Adults

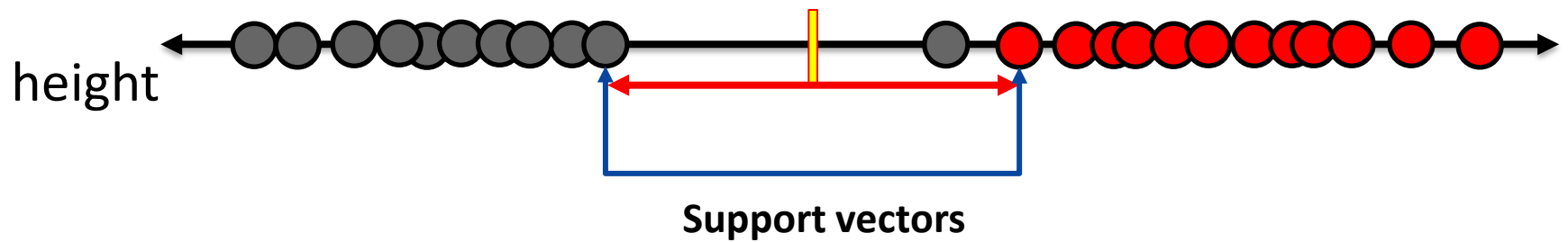
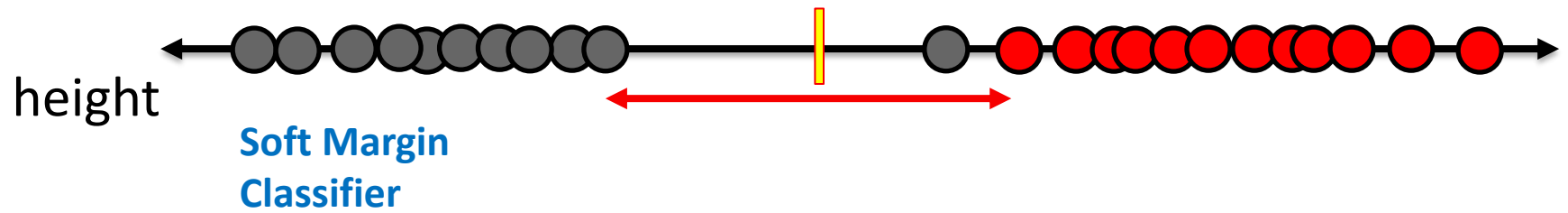
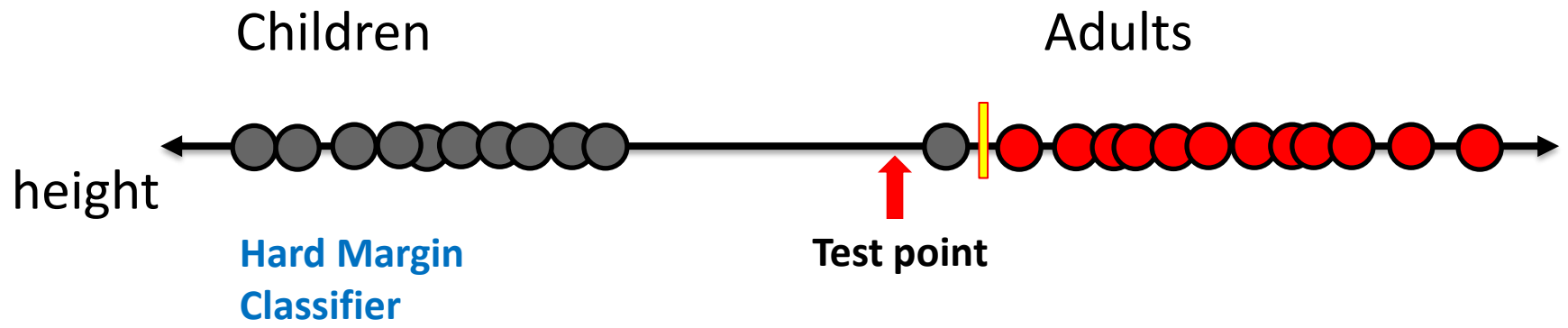


- If we want to place the hyperplane right in the middle of the nearest points from either side, then we want the two margins to be equal!
- In other words, we want to simultaneously maximize the margins on both sides.

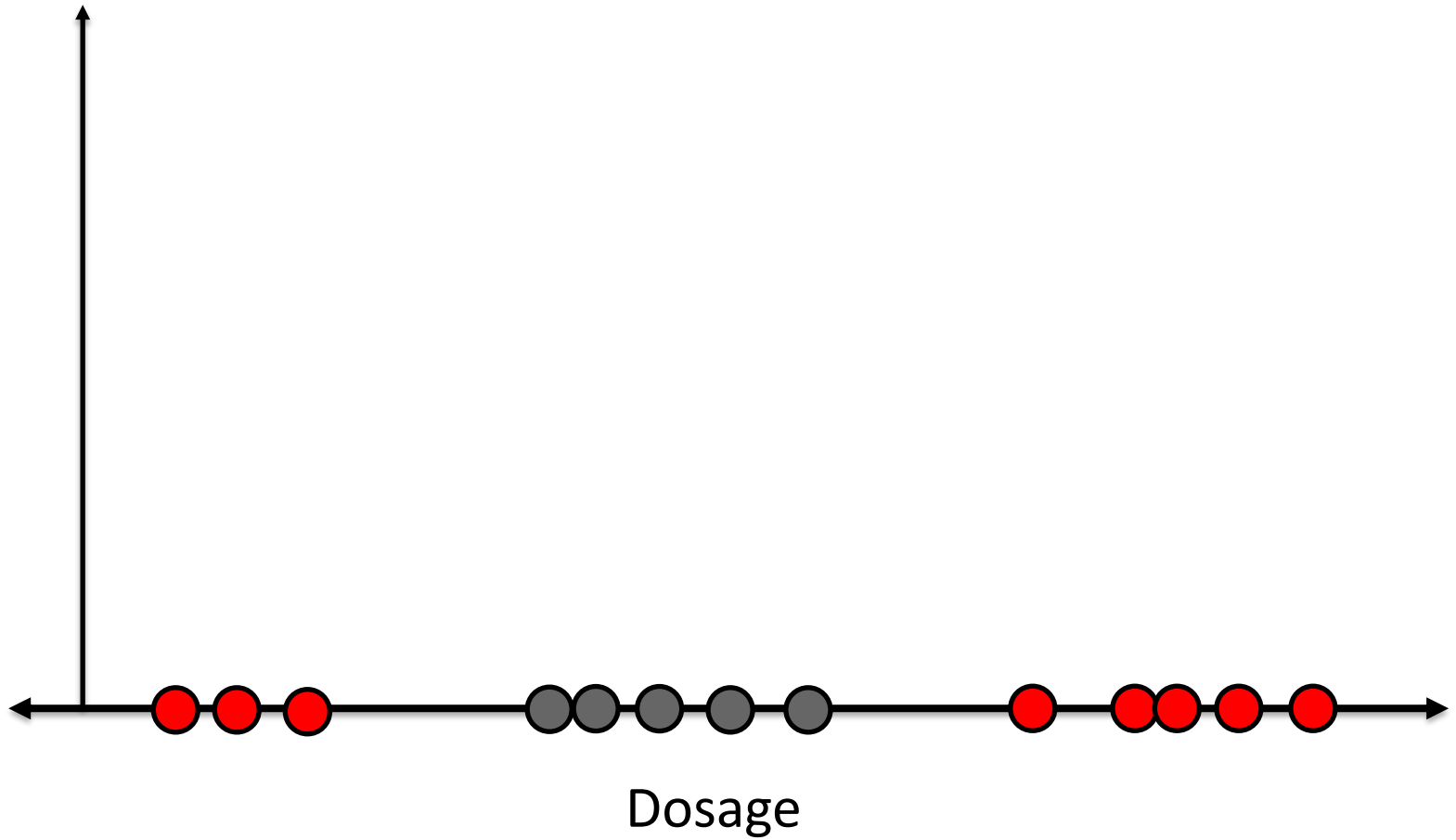


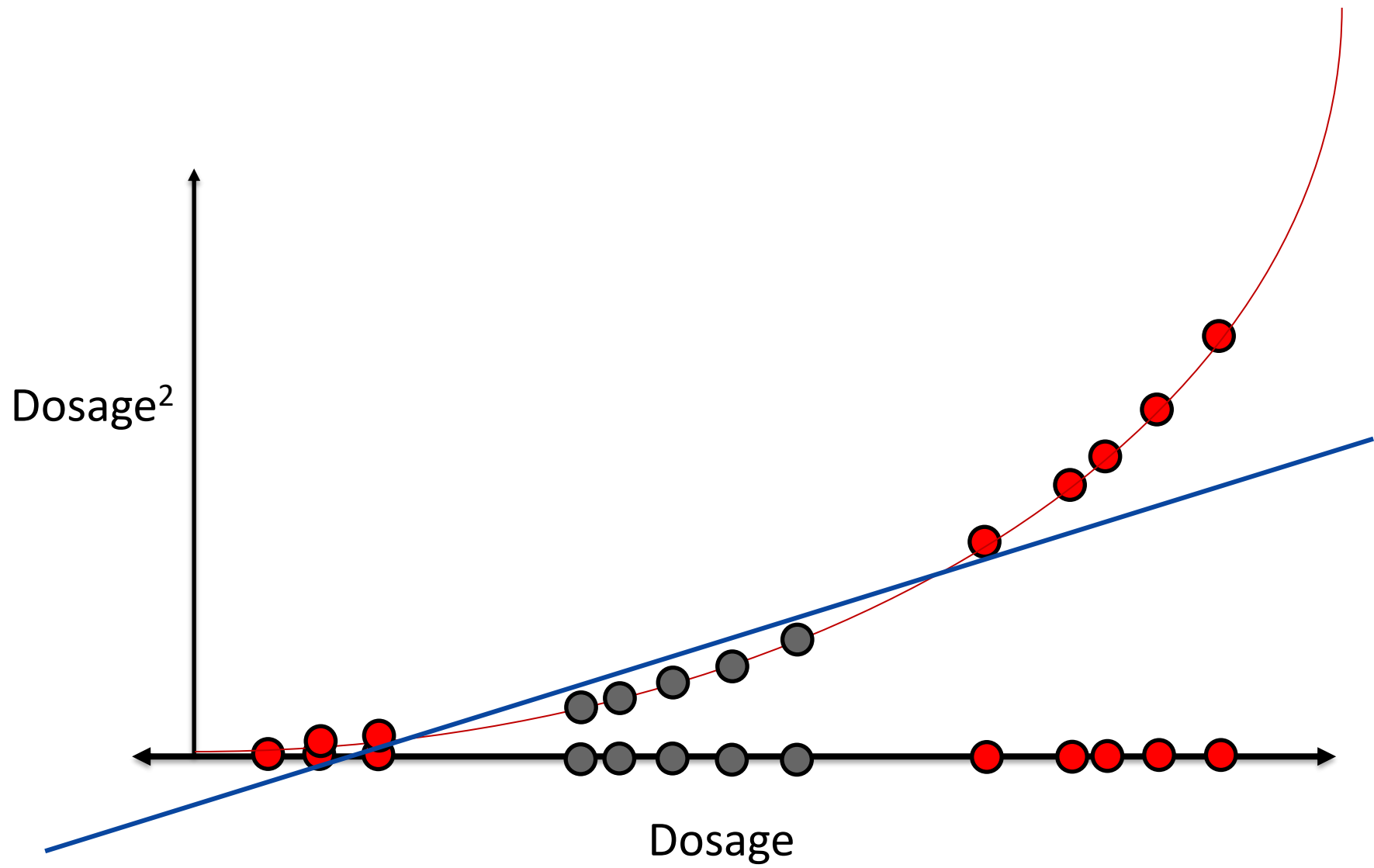
Maximum Margin Classifiers

That seems great!
But, is it always so?



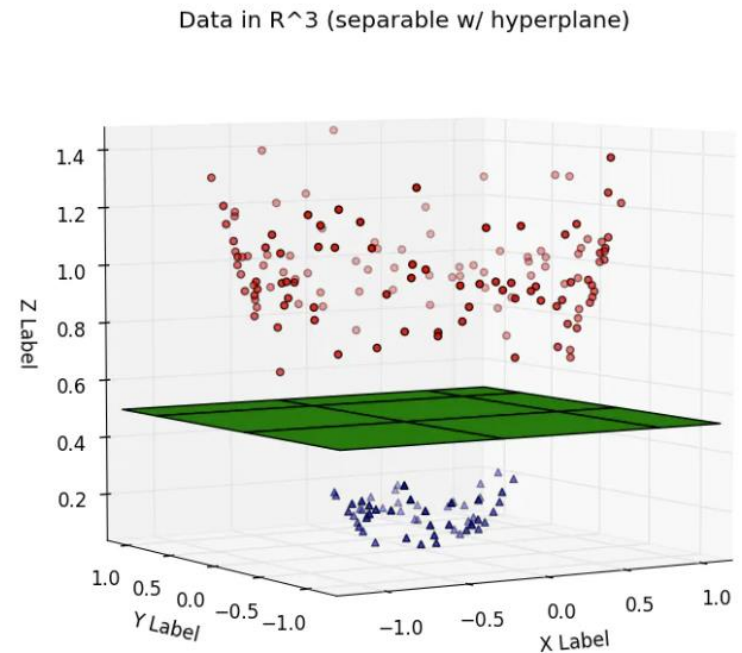
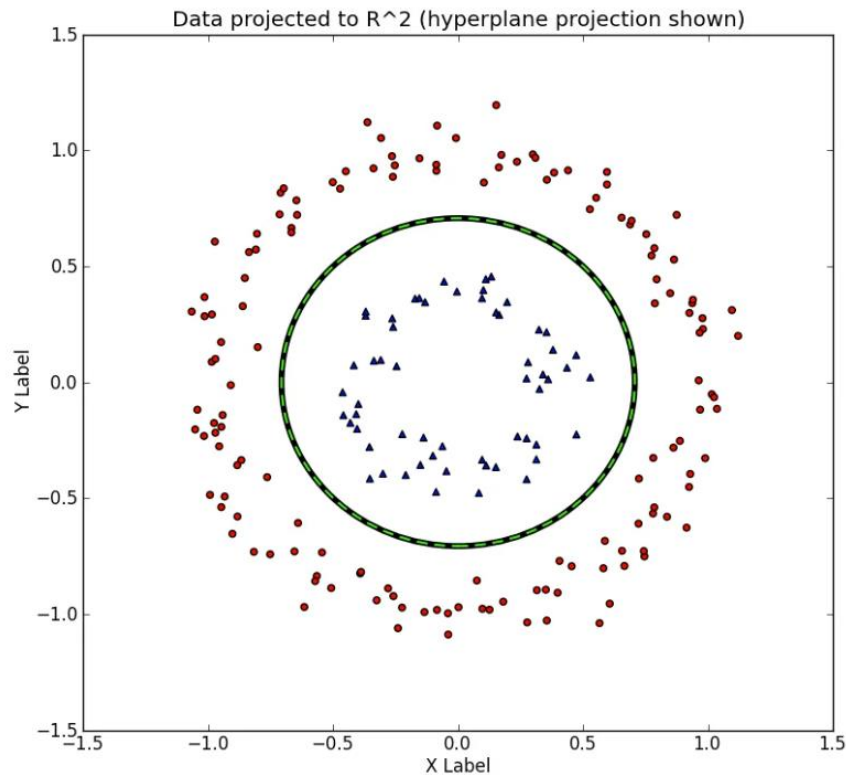
Not linearly Separable





Adding dimensions to achieve linear separability

(The Kernel Trick in Support Vector Classification, Drew Wilimitis,
<https://towardsdatascience.com/the-kernel-trick-c98cdbcaeb3f>)



Adding a third dimension: Each point's distance from the center

Linear Support Vector Machines

Hard Margin

Agha Ali Raza



Sources

- **Machine Learning for Intelligent Systems**, Kilian Weinberger, Cornell, Lecture 9,
https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecture_note09.html
- **Support Vector Machine: how it really works**,
<https://www.youtube.com/watch?v=A7FeQekjd9Q>, Victor Lavrenko, Assistant Professor at the University of Edinburgh
- **Support Vector Machine Python Example**, Cory Maklin,
<https://towardsdatascience.com/support-vector-machine-python-example-d67d9b63f1c8>
- **Support Vector Machine: Complete Theory**, Saptashwa Bhattacharyya, <https://towardsdatascience.com/understanding-support-vector-machine-part-1-lagrange-multipliers-5c24a52ffc5e>

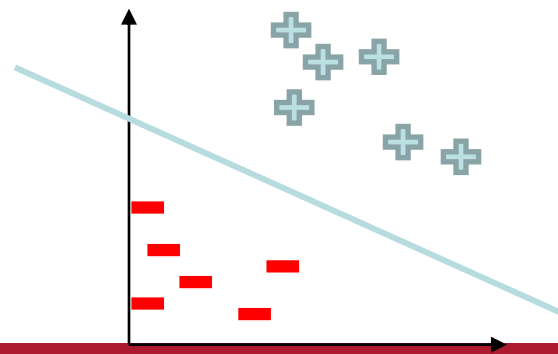
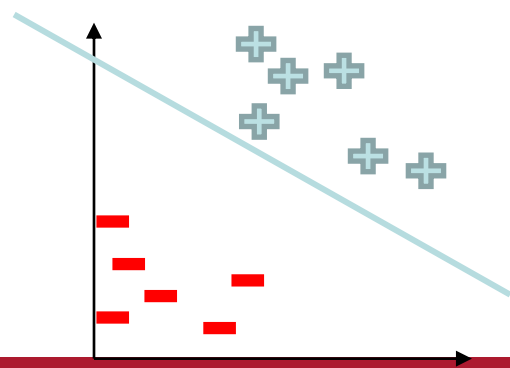
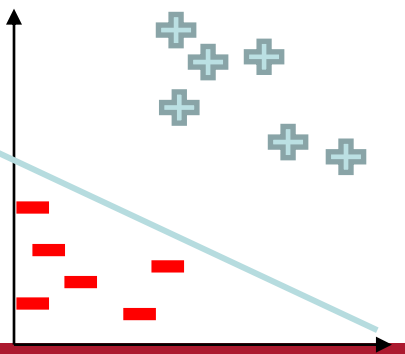
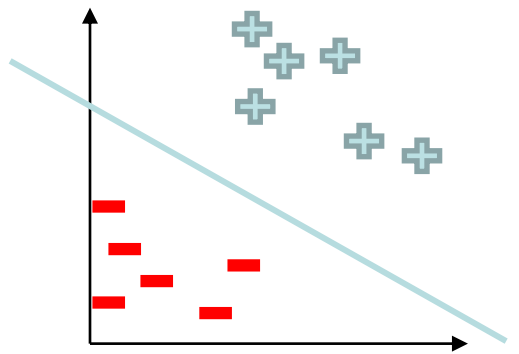
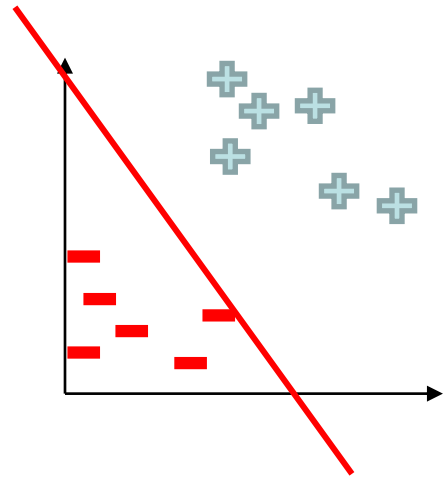
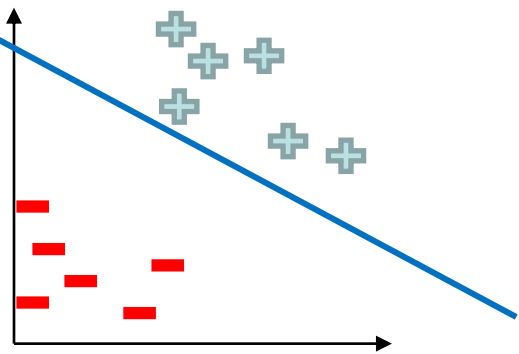
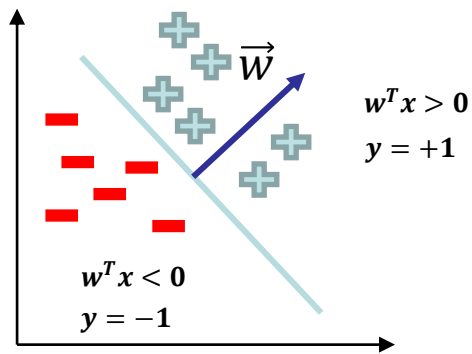
Try it yourself!

- Jupyter Notebooks from Andrew Ng's course: <https://github.com/Benlau93/Machine-Learning-by-Andrew-Ng-in-Python/tree/master/SupportVectorMachines>
- Blog post about the Jupyter Notebook: <https://towardsdatascience.com/andrew-ngs-machine-learning-course-in-python-support-vector-machines-435fc34b7bf9>

Support Vector Machines

(Machine Learning for Intelligent Systems, Kilian Weinberger, Cornell, Lecture 9)

The Perceptron



Support Vector Machines

(Machine Learning for Intelligent Systems, Kilian Weinberger, Cornell, Lecture 9)

The Support Vector Machine (SVM) is a linear classifier that can be viewed as an extension of the Perceptron developed by Rosenblatt in 1958.

- The Perceptron guaranteed that you find a hyperplane if it exists.
- The SVM finds the maximum margin separating hyperplane.

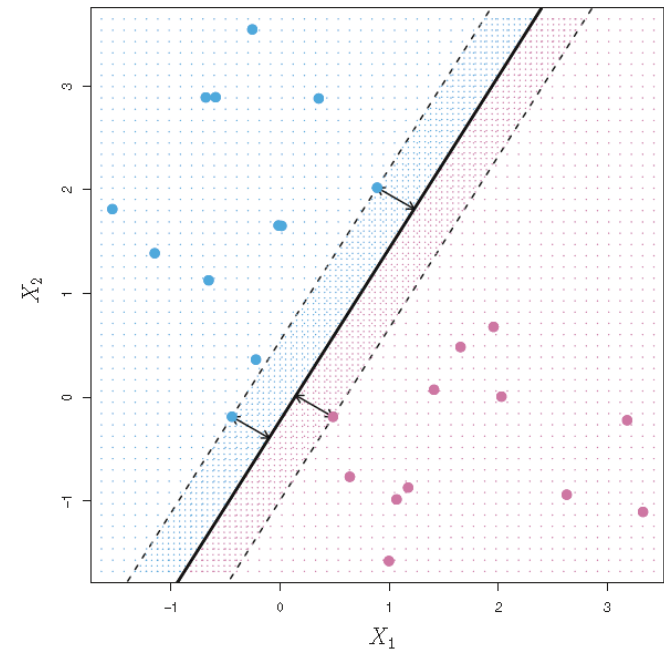
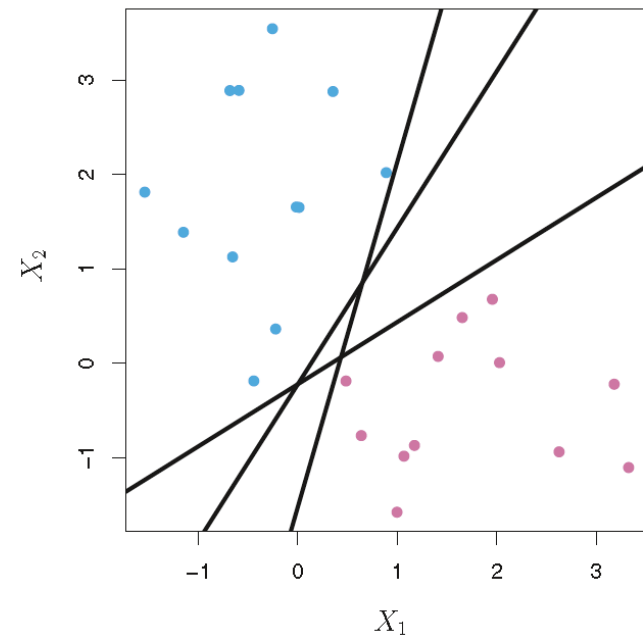


Image ref: <https://towardsdatascience.com/the-kernel-trick-c98cdbcaeb3f>

Support Vector Machines

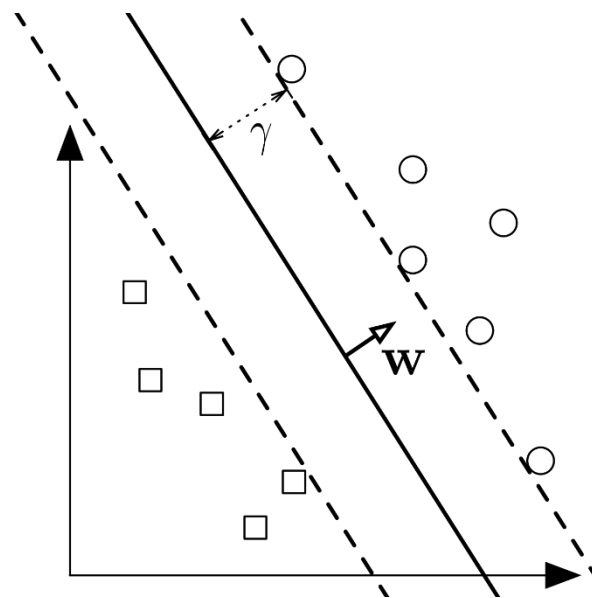
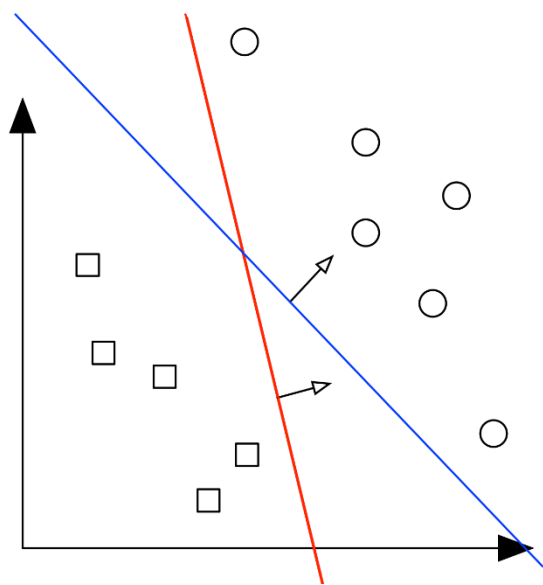
(Machine Learning for Intelligent Systems, Kilian Weinberger, Cornell, Lecture 9)

Setting: We define a linear classifier: $h(x) = \text{sign}(w^T x + b)$ and we assume a binary classification setting with labels $y \in \{+1, -1\}$.

If a data set is linearly separable, typically there are infinitely many separating hyperplanes. What is the best separating hyperplane?

SVM Answer: The hyperplane that maximizes the distance to the closest data points from both classes – the hyperplane with maximum margin

A hyperplane is defined as a set of points such that $H = \{x | w^T x + b = 0\}$. The margin γ is the distance from the hyperplane to the closest point across both classes.



Support Vector Machines

(<https://towardsdatascience.com/the-kernel-trick-c98cdbcaeb3f>)

- An optimization problem
- We maximize the margin: the distance separating the closest pair of data points belonging to opposite classes.
- These points are called the support vectors, because they are the data observations that “support”, or determine, the decision boundary.
 - These are also the samples that are most difficult to classify.
- To train an SVM classifier, we find the maximal margin hyperplane, or optimal separating hyperplane, which optimally separates the two classes in order to generalize to new data and make accurate classification predictions.

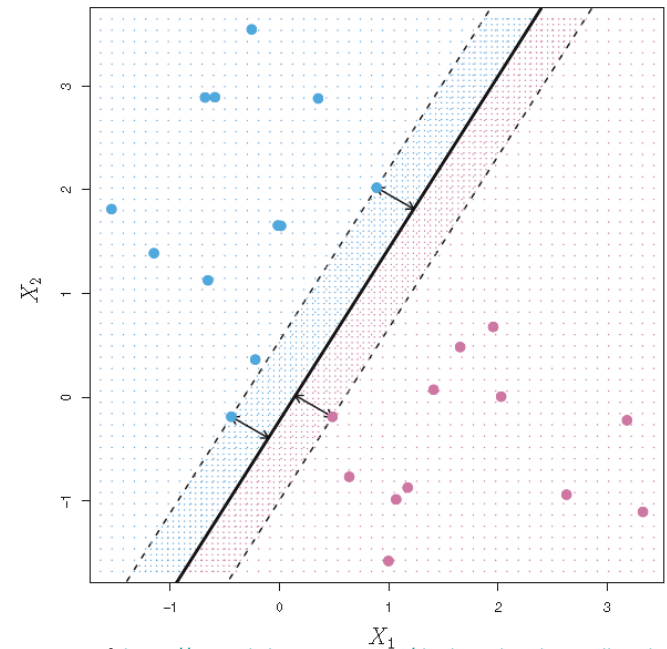


Image ref: <https://towardsdatascience.com/the-kernel-trick-c98cdbcaeb3f>

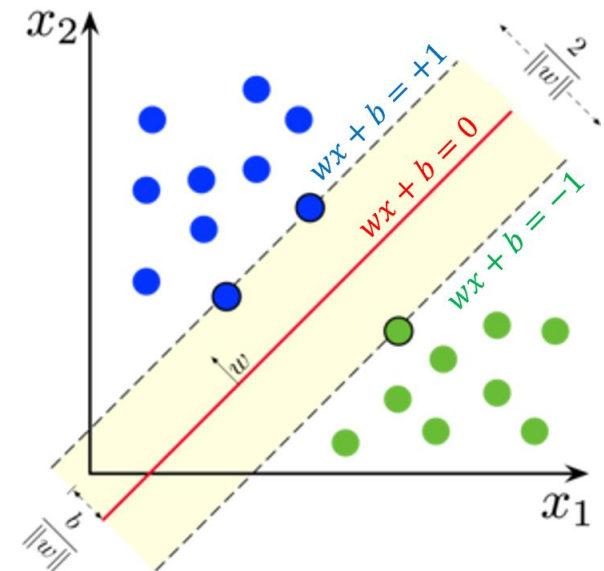


Image ref: https://en.wikipedia.org/wiki/Support-vector_machine

Support Vector Machines

(Machine Learning for Intelligent Systems, Kilian Weinberger, Cornell, Lecture 9)

- Remember that the (shortest) distance of a point x from a hyperplane H defined by the normal vector $\vec{w}$

$$\text{distance}(\vec{x}, \vec{w}) = |\vec{w} \cdot \vec{x}| \quad (\text{if } \vec{w} \text{ is a unit vector})$$

More generally,

$$\text{distance}(\vec{x}, \vec{w}) = \frac{|\vec{w} \cdot \vec{x}|}{\|\vec{w}\|_2} = \frac{|\vec{w} \cdot \vec{x} + b|}{\|\vec{w}\|_2} = \frac{|\vec{w}^T \vec{x} + b|}{\|\vec{w}\|_2}$$

There for the margin of H will be defined as:

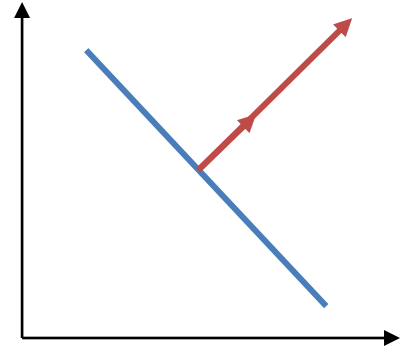
$$\gamma(w, b) = \min_{x \in D} \frac{|\vec{w}^T \vec{x} + b|}{\|\vec{w}\|_2}$$

By definition, the margin and hyperplane are scale invariant:

$$\gamma(\beta w, \beta b) = \gamma(w, b), \forall \beta \neq 0$$

- stretch $\vec{w}$ as much as you like, the plane will stay the same
- Increasing decreasing b will just move the plane around in n -dims

And shift the $n+1$ 'th dimension by a constant for the whole dataset in $n+1$ dims.



Support Vector Machines

(Machine Learning for Intelligent Systems, Kilian Weinberger, Cornell, Lecture 9)

Note: When γ is maximized, the hyperplane must lie right in the middle of the two classes

- Like a two-lane road!

γ must be the distance to the closest point within both classes.

- If not, the hyperplane could be moved towards data points of the class that is further away and increase γ , which contradicts that γ is maximized

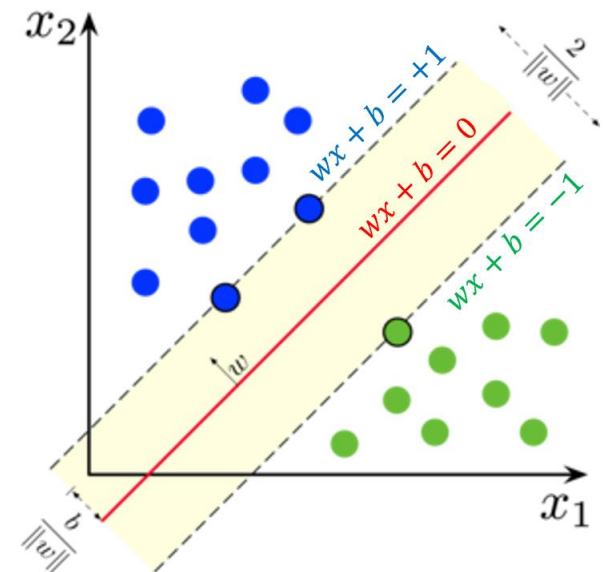


Image ref: https://en.wikipedia.org/wiki/Support-vector_machine

Formal Derivation

(Machine Learning for Intelligent Systems, Kilian Weinberger, Cornell, Lecture 9)

- We can formulate our search for the maximum margin separating hyperplane as a constrained optimization problem.
- The objective is to maximize the margin under the constraints that all data points must lie on the correct side of the hyperplane:

$$\max_{w,b} \gamma(w,b) \quad s.t. \quad \forall i \, y_i(w^T x_i + b) \geq 0$$

Maximize margin

Separating hyperplane

- If we plug in the definition of γ we obtain:

$$\max_{w,b} \left(\min_{x \in D} \frac{|\vec{w}^T \vec{x} + b|}{\|\vec{w}\|_2} \right) \text{ such that } \forall i \, y_i(w^T x_i + b) \geq 0$$

- As $\|\vec{w}\|_2$ will not be impacted by the $\min_{x \in D}$, taking it outside:

$$\max_{w,b} \frac{1}{\|\vec{w}\|_2} \left(\min_{x \in D} |\vec{w}^T \vec{x} + b| \right) \text{ such that } \forall i \, y_i(w^T x_i + b) \geq 0$$

Formal Derivation

(Machine Learning for Intelligent Systems, Kilian Weinberger, Cornell, Lecture 9)

$$\max_{w,b} \frac{1}{\|\vec{w}\|_2} \left(\min_{x \in D} |\vec{w}^T \vec{x} + b| \right) \text{ such that } \forall i \ y_i (w^T x_i + b) \geq 0$$

Maximize margin

Separating hyperplane

- Because the hyperplane is scale invariant, we can fix the scale of w, b anyway we want:

$$\min_{x \in D} |\vec{w}^T \vec{x} + b| = 1$$

- We can add this re-scaling as an equality constraint. Then our objective becomes:

$$\max_{w,b} \frac{1}{\|w\|_2} \cdot 1 = \min_{w,b} \|w\|_2 = \min_{w,b} w^T w$$

- We have ignored the square-root in $\sqrt{w^T w}$, as $f(z) = z^2$ is a monotonically increasing function for $z \geq 0$ and $\|w\| \geq 0$; i.e. the w that maximizes $\|w\|_2$ also maximizes $w^T w$

Formal Derivation

(Machine Learning for Intelligent Systems, Kilian Weinberger, Cornell, Lecture 9)

- Our new optimization problem becomes:

$$\begin{aligned} & \min_{w,b} w^T w \\ \text{s.t. } & \forall i, y_i (w^T x_i + b) \geq 0 \\ & \min_i |w^T x_i + b| = 1 \end{aligned}$$

- These constraints are still hard to deal with, however luckily we can show that (for the optimal solution) they are equivalent to a much simpler formulation.

$$\begin{aligned} & \min_{w,b} w^T w \\ \text{s.t. } & \forall i, y_i (w^T x_i + b) \geq 1 \end{aligned}$$

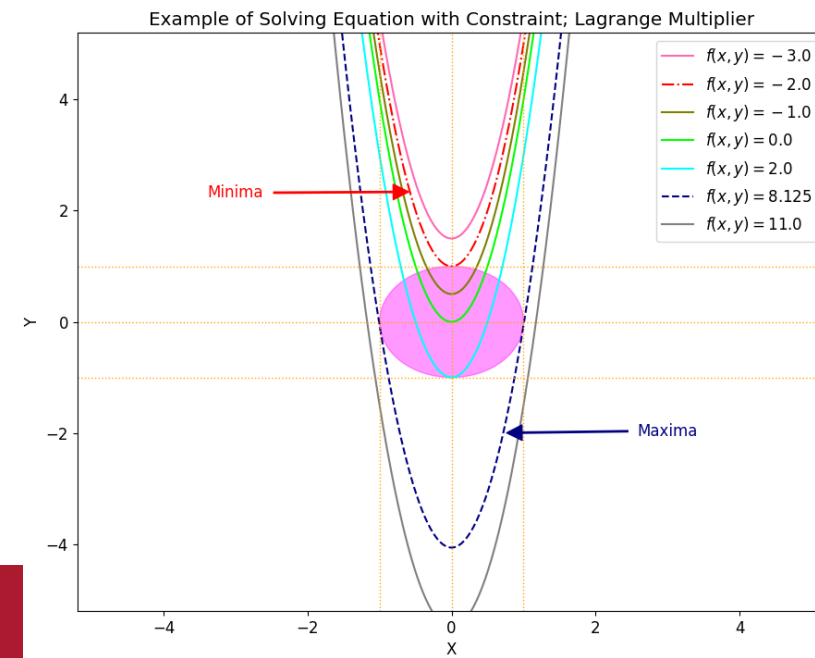
- Find the simplest hyperplane (where simpler means smaller $w^T w$) such that all inputs lie at least 1 unit away from the hyperplane on the correct side
- This is a linearly constrained quadratic optimization problem, that could be solved using quadratic programming e.g. using Lagrange Multipliers.

Constrained Optimization and Lagrange Multipliers

(<https://towardsdatascience.com/understanding-support-vector-machine-part-1-lagrange-multipliers-5c24a52ffc5e>)

- Suppose we are given a function $f(x, y, z, \dots)$ for which we want to find the extrema, subject to the condition $g(x, y, z, \dots) = k$
- The idea used in Lagrange multiplier is that the gradient of the objective function f , lines up either in parallel or anti-parallel direction to the gradient of the constraint g , at an optimal point. In such case, one of the gradients should be some multiple of another.
- E.g. $f(x, y) = 8x^2 + 2y$, $g(x, y) = x^2 + y^2$
$$\nabla f = \lambda \nabla g$$

- The extrema of constrained function f , lie on the surface of the constraint g , which is a circle of unit radius.
- It is a necessary condition.
- The tangent vectors of the function and the constraint are either parallel or anti-parallel at each extremum.



SVMs

(Machine Learning for Intelligent Systems, Kilian Weinberger, Cornell, Lecture 9)

- For the optimal w, b pair, some training points will have tight constraints, i.e.

$$y_i(w^T x_i + b) = 1$$

- This must be the case, because if for all training points we had a strict $>$ inequality, it would be possible to scale down both parameters w, b until the constraints are tight and obtained an even lower objective value
- We refer to these training points as **support vectors**
 - Support vectors are special because they are the training points that define the maximum margin of the hyperplane to the data set and they therefore determine the shape of the hyperplane.
 - If you were to move one of them and retrain the SVM, the resulting hyperplane would change.
 - The opposite is the case for non-support vectors (provided you don't move them too much, or they would turn into support vectors themselves).
 - This will become particularly important in the dual formulation for Kernel-SVMs.

Up next

- Linear SVMs with soft margins

Linear Support Vector Machines

Soft Margin

Agha Ali Raza



Sources

- **Machine Learning for Intelligent Systems**, Kilian Weinberger, Cornell, Lecture 9,
<https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecturenote09.html>
- **Support Vector Machine: how it really works**,
<https://www.youtube.com/watch?v=A7FeQekjd9Q>, Victor Lavrenko, Assistant Professor at the University of Edinburgh
- **Support Vector Machine Python Example**, Cory Maklin,
<https://towardsdatascience.com/support-vector-machine-python-example-d67d9b63f1c8>
- **Support Vector Machine: Complete Theory**, Saptashwa Bhattacharyya,
<https://towardsdatascience.com/understanding-support-vector-machine-part-1-lagrange-multipliers-5c24a52ffc5e>
- **Support Vector Machine: Digit Classification with Python; Including my Hand Written Digits**, Saptashwa Bhattacharyya,
<https://towardsdatascience.com/support-vector-machine-mnist-digit-classification-with-python-including-my-hand-written-digits-83d6eca7004a>
- **Support Vector Machine: Kernel Trick; Mercer's Theorem**, Saptashwa Bhattacharyya, <https://towardsdatascience.com/understanding-support-vector-machine-part-2-kernel-trick-mercers-theorem-e1e6848c6c4d>

Hard Margin Support Vector Classifier

(Machine Learning for Intelligent Systems, Kilian Weinberger, Cornell, Lecture 9)

$$\begin{aligned} \min_{w,b} \quad & w^T w \\ \text{s.t.} \quad & \forall i, y_i (w^T x_i + b) \geq 0 \\ & \min_i |w^T x_i + b| = 1 \end{aligned}$$

- Hard constraints! Luckily, we have an equivalent formulation for the optimal solution:

$$\begin{aligned} \min_{w,b} \quad & w^T w \\ \text{s.t.} \quad & \forall i, y_i (w^T x_i + b) \geq 1 \end{aligned}$$

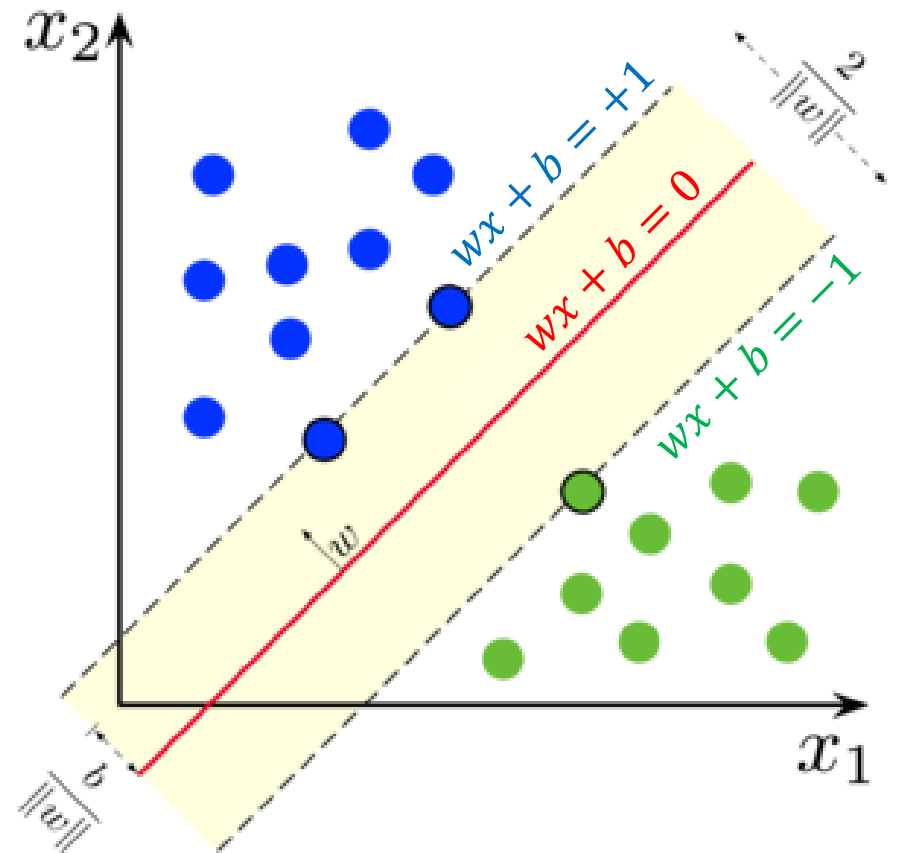
- This is a linearly constrained quadratic optimization problem, that could be solved using quadratic programming e.g. using Lagrange Multipliers.

Some Intuition:

$$\begin{aligned} & \min_{w,b} w^T w \\ & \text{s.t. } \forall i \ y_i(w^T x_i + b) \geq 1 \end{aligned}$$

$$\begin{aligned} & \min_{w,b} \|w\|_2 \\ & \text{s.t. } \forall i \ y_i(w^T x_i + b) \geq 1 \end{aligned}$$

We are looking for the widest road that allows linear separation of the classes!



Soft Margin SVMs (Machine Learning for Intelligent Systems, Kilian Weinberger, Cornell, Lecture 9)

- For low-dimensional data there might be no separating hyperplane between the two classes.
- In this case, there is no solution to the optimization problems for the hard margin SVMs.
- **Solution:** Allow the constraints to be slightly violated using slack variables:

$$\begin{aligned} \min_{w,b} \quad & w^T w + C \sum_{i=1}^n \xi_i \\ \text{s.t.} \quad & \forall i \ y_i (w^T x_i + b) \geq 1 - \xi_i \\ & \forall i \ \xi_i \geq 0 \end{aligned}$$

- The slack variable ξ_i allows the input x_i to be closer to the hyperplane (or even be on the wrong side)
 - There is a penalty in the objective function for such "slack"
 - **If C is very large**, the SVM becomes very strict and tries to get all points to be on the correct side of the hyperplane
 - **If C is very small**, the SVM becomes very lenient and may sacrifice some points to obtain a simpler (i.e. lower $\|w\|_2$) solution.

Unconstrained Formulation

(Machine Learning for Intelligent Systems, Kilian Weinberger, Cornell, Lecture 9)

- Assuming $C \neq 0$

$$\xi_i = \begin{cases} 1 - y_i(w^T x_i + b) & \text{if } y_i(w^T x_i + b) < 1 \\ 0 & \text{if } y_i(w^T x_i + b) \geq 1 \end{cases}$$

- Which is equivalent to:

$$\xi_i = \max(1 - y_i(w^T x_i + b), 0)$$

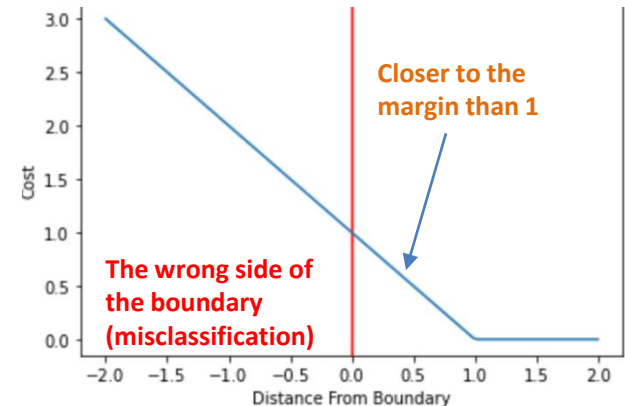
- Plugging this closed form into the objective of our SVM optimization problem, we obtain the following unconstrained version as loss function and regularizer:

$$\min_{w,b} w^T w + C \sum_{i=1}^n \max(1 - y_i(w^T x_i + b), 0)$$

- This formulation allows us to optimize the SVM parameters (w, b) just like logistic regression (e.g. through gradient descent)
- The only difference is that we have hinge-loss instead of the log loss

The Hinge Loss

- The hinge loss is a specific type of cost function that incorporates **a margin or distance from the classification boundary** into the cost calculation.
 - Even if new observations are classified correctly, they can still incur a penalty if the margin from the decision boundary is not large enough.
 - The hinge loss increases linearly.
- **Support vector machines** address a classification problem where observations either have an outcome of +1 or -1.
 - The SVM produces a real-valued output that is negative or positive depending on which side of the decision boundary it falls.
 - Only if an observation is classified correctly and the distance from the plane is larger than the margin will it incur no penalty.
 - The distance from the hyperplane can be regarded as a measure of confidence. The further an observation lies from the plane, the more confident it is in the classification.



The Hinge Loss

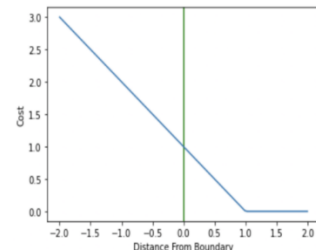
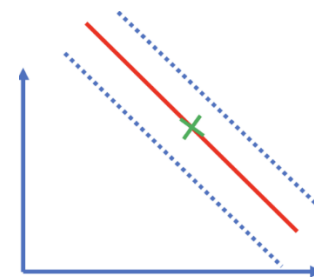
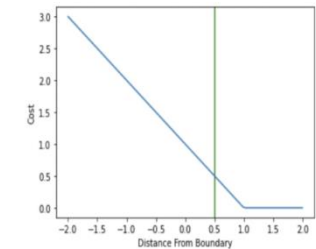
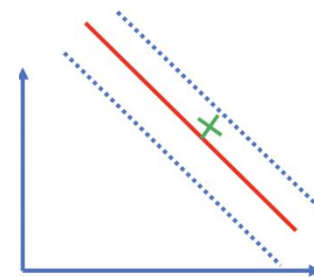
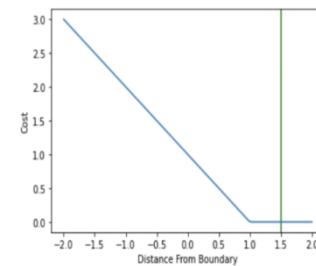
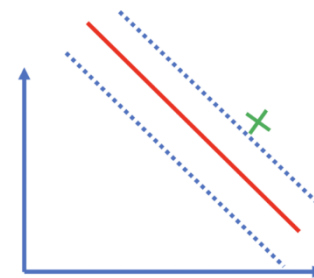
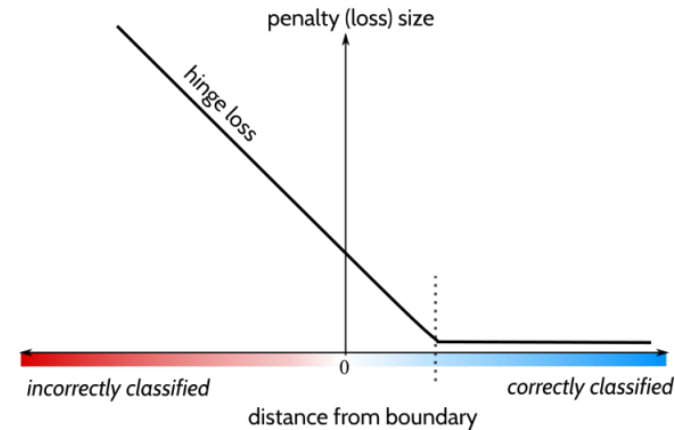
$$\text{Hinge Loss} = \xi_i = \begin{cases} 1 - y_i(w^T x_i + b) & \text{if } y_i(w^T x_i + b) < 1 \\ 0 & \text{if } y_i(w^T x_i + b) \geq 1 \end{cases}$$

Remember:

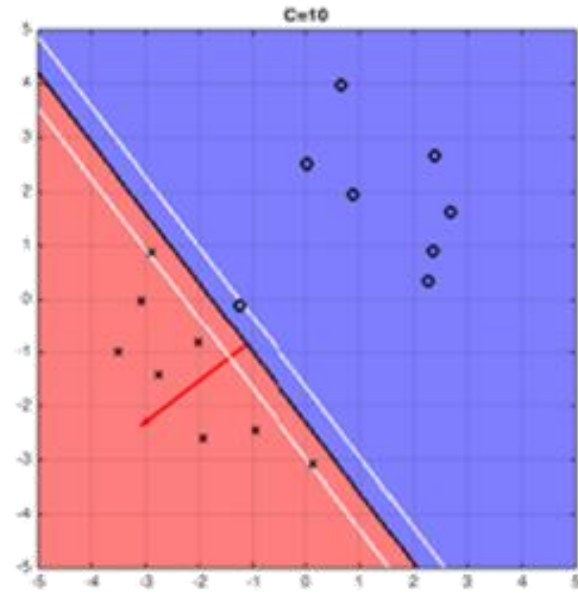
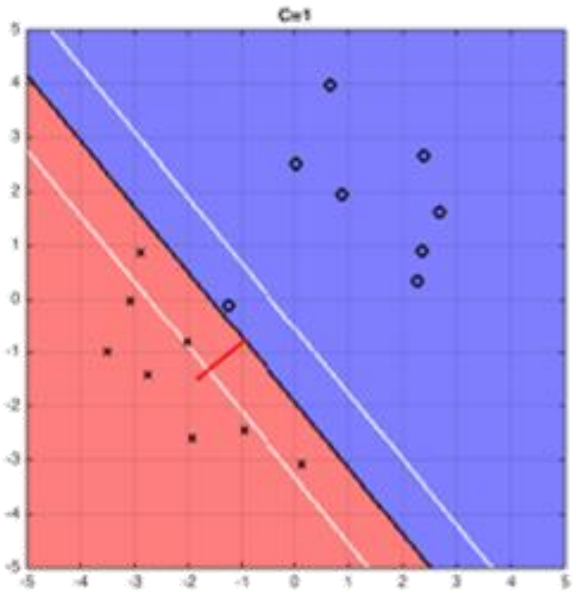
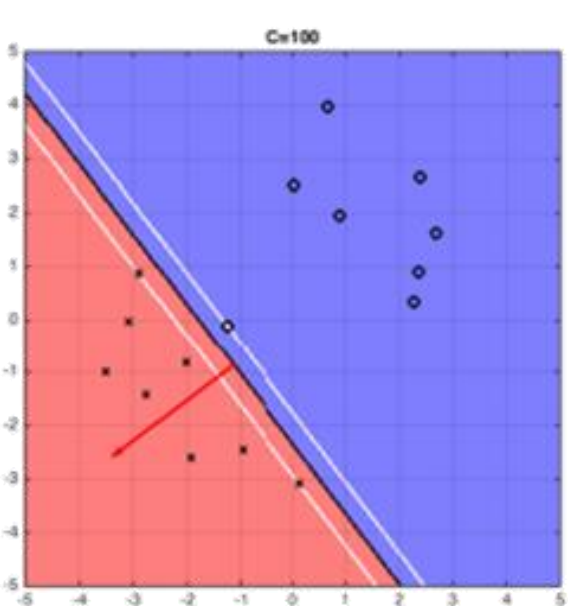
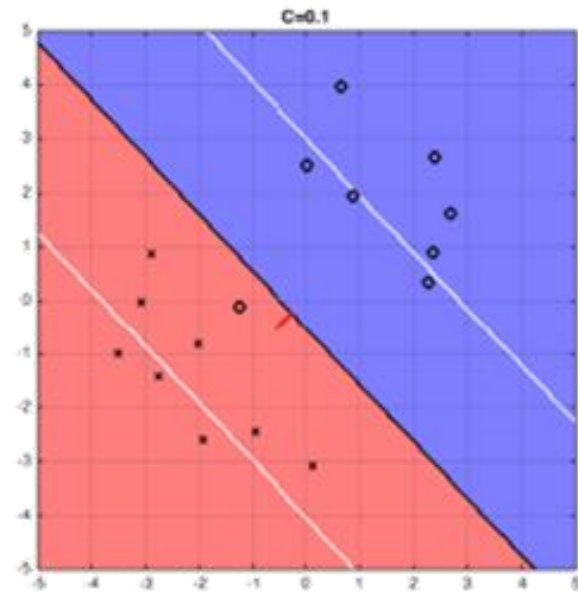
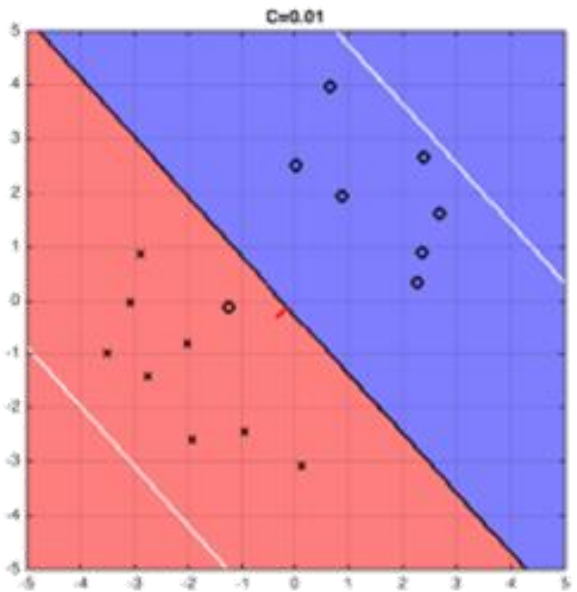
- If y_i and $(w^T x_i + b)$ have the same sign, it is a correct classification and the product $y_i(w^T x_i + b)$ would be positive
 - This could go under $y_i(w^T x_i + b) < 1$ or $y_i(w^T x_i + b) \geq 1$, and would produce a low loss (0) when the magnitude of the product is higher than 1 or a higher loss ($1 - y_i(w^T x_i + b)$), otherwise
- If y_i and $(w^T x_i + b)$ have different signs, it is an incorrect classification and the product $y_i(w^T x_i + b)$ would be negative
 - This would go under $y_i(w^T x_i + b) < 1$ and would produce loss $1 - y_i(w^T x_i + b)$, which would scale linearly depending on the magnitude of the product

The above is equivalent to:

$$\text{Hingeloss} = \xi_i = \max(1 - y_i(w^T x_i + b), 0)$$



(Machine Learning for Intelligent Systems, Kilian Weinberger, Cornell, Lecture 9)



For more details please visit
<http://aghaaliraza.com>



Thank you!